

Threshold Signatures from One-Way Functions

Pedro Branco
Bocconi University

Giulio Malavolta
Bocconi University

Abstract

A threshold signature allows one to delegate its signing rights to n parties, such that any subset of size t can sign a message on their behalf. In this work, we show how to construct threshold signatures for any t and n from one way functions, thus establishing the latter as a necessary and sufficient computational assumption. Our protocol makes non-black box use of one-way functions, and can be generalized to other access structures, such as monotone policies.

1 Introduction

Threshold signatures [Des88, DF90] allow one to share the secret key of a signature scheme across several parties. Later, a subset of these parties can come together and interact to compute a valid signature on a given message, provided that their size reaches a certain threshold. Although the concept of threshold signatures has existed since the 1980s, it gained renewed interest due to its applications in crypto wallets [GGN16, AB23] and the prospect of a NIST standardization of this notion [NIS, CKGW23].

Due to their large applicability, threshold signatures are among the most well-studied topics in cryptography today. They have been explored in many settings, such as interactive [KG20, TZ23] and non-interactive [BLS01, Bol03, BGG⁺18, ASY22] signing, no trusted setup [BGJ⁺23, GJM⁺24], and with enhanced security guarantees [RRJ⁺22, CKM23, BLT⁺24]. We do not attempt to survey all works on threshold signatures here, and we refer the reader to, e.g., [Kom24, KRT24, CATZ25] and references therein.

Despite this progress, our understanding of the *minimal assumptions* required to construct threshold signatures remains limited. To the best of our knowledge, all known constructions of threshold signatures with a security proof in the standard model rely on *structured algebraic assumptions*. This stands in stark contrast to standard signatures, which have long been known to be implied by one-way functions (OWFs) [Lam79, NY89, Rom90]. Given this state of affairs, the central question that we address in this work is:

What are the minimal assumptions required to build threshold signatures?

1.1 Our Results

Before stating our results, let us clarify the specifics of the settings we consider. We consider the scenario where: (i) A trusted setup distributes a secret key to n parties, then (ii) any set of t such parties can interact with each other to generate a signature. (iii) Security requires that an adversary corrupting any set of size at most $t - 1$ cannot forge a valid signature, even if it is allowed to behave arbitrarily during the interactive protocol. This setting is standard in threshold signature research and has been adopted in most recent works, e.g., [KG20, RRJ⁺22, TZ23, CKM23, BLT⁺24].

In this setting, we show that it is possible to construct threshold signatures from any one-way function. Given that threshold signatures imply regular signatures, and that the latter imply the existence of one-way functions, we obtain a complete characterization of this primitive. That is, we show that

$$\text{Threshold Signatures} \equiv \text{One Way Functions.}$$

This closes the gap in our understanding of the minimal assumptions required for threshold signatures, bringing them in line with those needed for standard signatures. In fact, our result is more general and shows a method to *compile* any regular signature scheme into a threshold one, for any polynomial number of parties n , and any threshold $t \leq n$.

Our result can be easily generalized to support not only threshold policies but also any monotone-policy signatures [BCJP24]: In a monotone policy signature scheme, shares are generated according to a policy P that can be expressed as a monotone program. An authorized subset S of signers is one for which $P(S) = 1$, meaning that only such subsets can issue valid signatures.

1.2 Technical Overview

To set some context for our solution, let us recall some folklore approaches to build threshold signatures and why they fail to provide us with a scheme from one-way functions.

Simple Approaches and Why They Fail. The simplest approach to construct a threshold signature is the obvious one: Set the verification key vk as the concatenation of N independent vk_i where each party has a secret key for one of them. When a subset S of t parties wants to sign a message, they simply issue a signature σ_i under the corresponding secret key for the message m . The signature is composed of $\{\sigma_i\}_{i \in S}$. On the positive side, we only require the existence of a signature scheme, which is known to be implied by one-way functions. On the other hand, the verification key and signatures grow with the number of users, which is disallowed by the efficiency requirement of a threshold signature.

Another approach is to use succinct arguments of knowledge (SNARKs). As above, each of the t parties signs a message using its secret key. One can then later combine all of these signatures with a SNARK for NP, proving the knowledge of t signatures on the message. One problem with this approach is that the signature is now clearly distinguishable from the original one and, more severely, that SNARKs are only known to exist either in the

random oracle model or using non-falsifiable assumptions. Another approach would be to use a monotone-policy SNARG [BBK⁺23, BCJP24, NWW25], which however is only known from assumptions implying public-key cryptography.

Finally, the last folklore construction of threshold signature is via multiparty computation (MPC) protocols. Here, the secret key of a signature scheme is first secret shared across all parties using a t -out-of- n scheme. To sign a message m , the parties engage in an MPC for the circuit that reconstructs the secret key from the shares and signs m . Alas, it is well-known that MPC with dishonest majority implies the existence of an oblivious transfer (OT) protocol, and it is therefore unlikely to be built from one-way functions [IR89].

One-Time Threshold Signatures from OWF. Our first observation is that we can salvage some ideas from the last approach. Let us consider for the moment the toy setting where security is required only for a *single* run of the signing protocol, i.e., with a single subset of parties signing a single message. We observe that in this setting, we can indeed use the above idea to build threshold signatures from one-way functions: We let the trusted setup provide the parties with sufficiently many OT correlations, that can be later consumed to perform a run of the MPC. Equipped with OT correlations, one-way functions are then sufficient to securely compute any function [IPS08]. Concretely, the protocol goes as follows.

- (Setup) Each party receives a secret share share_i of the secret key sk for a signature scheme. Additionally, each pair of parties (P_i, P_j) receive a fixed number of OT correlations $(\text{ot}_{R,i,j}, \text{ot}_{S,j,i})$ for all $i \neq j$. Here $\text{ot}_{R,i,j}$ denotes the receiver OT correlation, held by party P_i , and $\text{ot}_{S,j,i}$ denotes the sender OT correlation, held by party P_j .
- (Signing) When a subset of parties S wants to sign a message m , they run an MPC using the OT correlations for the circuit that does the following: It receives the shares $\{\text{share}_i\}_{i \in S}$, reconstructs sk and signs the message m using sk .

Security of the threshold signature scheme follows immediately from the security of the underlying MPC. Moreover, all the building blocks used are implied by the existence of one-way functions. On the other hand, OT correlations are *not reusable* (else, security of the MPC would be violated) hence this approach seems to be fundamentally stuck with a single signing protocol.

OT Extensions. To extend one-time signatures to many-time signatures, our idea is to use OT extension protocols [Bea96, IKNP03] to synthesize more OT correlations out of a (small) fixed number of base OTs. Importantly for us, OT extension protocols can be realized using one-way functions only [Bea96]. Using this tool, we can lift the previous approach to signing many messages, without changing the size of the signing key. Unfortunately, a naive application of this idea has several problems:

- Once we extend the OT correlations, the base OT correlations cannot be reused and in particular they cannot be re-extended. This implies that there is a bound on the total number of signatures.

- Each user needs to be a stateful algorithm. This is because, after extending the OT correlations, each party needs to remember which pairs of OT correlations were already used and which were not.

A New Primitive: Tree OT Extensions. To solve the above problems, we will introduce a new primitive called *Tree OT extensions*. In a tree OT extension, two parties P_1 and P_2 start with a fixed number of OT correlations $(\text{ot}_{R,1,2}, \text{ot}_{S,2,1})$. On input a string S of polynomial size, parties interact and are able to create a new set of OT correlations $(\text{ot}_{R,1,2}(S), \text{ot}_{S,2,1}(S))$. Crucially, we want the following properties to hold:

- If parties run the protocol twice on the same input S , they get the same set of OT correlations $(\text{ot}_{R,1,2}(S), \text{ot}_{S,2,1}(S))$.
- The base OT correlations $(\text{ot}_{R,1,2}, \text{ot}_{S,2,1})$, can be reused to obtain a new set of OT correlations $(\text{ot}_{R,1,2}(S'), \text{ot}_{S,2,1}(S'))$ for a new string S' .

Armed with this new primitive, we can revisit the previous approach to obtain a threshold signature scheme in the following way:

- (Setup) Each party receives a secret share share_i of the secret key sk for a signature scheme. Additionally, each pair of parties (P_i, P_j) receives a fixed number of OT correlations $(\text{ot}_{R,i,j}, \text{ot}_{S,j,i})$ for all $i \neq j$.
- (Signing) When a subset of parties S wants to sign a message m , they do the following:
 - Each pair of parties P_i and P_j runs the tree OT extension to obtain a new set of OT correlations $(\text{ot}_{R,i,j}(S||m), \text{ot}_{S,j,i}(S||m))$ for all $i, j \in S$ such that $i \neq j$.
 - They run an MPC, using the OT correlations obtained from the previous stage, to securely compute the circuit that does the following: It receives the shares $\{\text{share}_i\}_{i \in S}$, reconstructs sk and signs the message m using sk .

Tree OT extensions from OWFs. It remains to show how we can build tree OT extensions from OWFs. Our construction can be seen as combining two classical techniques from cryptography: Beaver’s OT extension [Bea96] and the Naor-Yung paradigm [NY89]. We present here a simplified version of our scheme, and we refer the reader to the technical sections for more details. For a string $S = (S_1, \dots, S_k)$, $S_{\leq i}$ denotes the string $(S_1, \dots, S_i)$.

- (Setup) The parties P_1 and P_2 receive a set of OT correlations $(\text{ot}_{R,1,2}(\varepsilon), \text{ot}_{S,2,1}(\varepsilon))$.
- On input a string $S = (S_1, \dots, S_k)$, parties do the following for all $i = 1, \dots, k - 1$:
 - Each party parses $\text{ot}_{\alpha,i,j}(S_{\leq i})$ as $(\text{ot}_{\alpha,i,j,0}(S_{\leq i}) || \text{ot}_{\alpha,i,j,1}(S_{\leq i}))$, that is, each set of correlations is parsed as a left set $\text{ot}_{\alpha,i,j,0}(S_{\leq i})$ and a right set $\text{ot}_{\alpha,i,j,1}(S_{\leq i})$.
 - Parties use Beaver’s OT extension to extend $(\text{ot}_{R,1,2,S_i}(S_{\leq i}), \text{ot}_{S,2,1,S_i}(S_{\leq i}))$ into a new set $(\text{ot}_{R,1,2}(S_{\leq i+1}), \text{ot}_{S,2,1}(S_{\leq i+1}))$. That is, depending on the i -th bit of S , at level i of the tree they extend either the left or the right set of OT correlations.

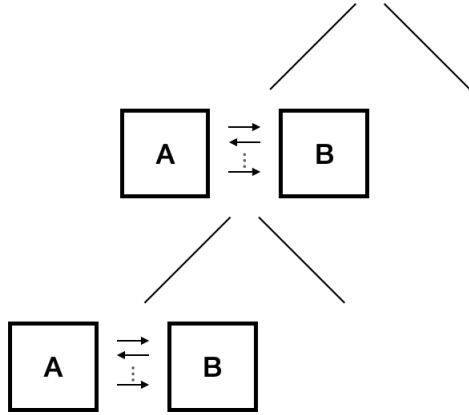


Figure 1: Tree OT extensions. At each level i of the tree, parties decide either to extend the left set or the right set of OT correlations, depending on the i -th bit of S .

- Parties output the set of OT correlations $(\text{ot}_{R,1,2}(S), \text{ot}_{S,2,1}(S))$.

An illustration of the process is given in Fig. 1. Security for one execution of the protocol follows from the security of the Beaver’s OT extension protocol. For the protocol to be secure for many queries S , we need parties to be stateful since we cannot guarantee security if the same set of OT correlations is used on two different inputs. Finally, to go from stateful to stateless, each party generates their random coins pseudorandomly from a short seed, once again using only one-way functions.

1.3 Open Problems

Our work is a stepping stone towards understanding the assumptions needed for threshold signatures. However, our work still leaves open questions such as constructing threshold signatures with non-interactive signing from OWF, or making only black box use of the OWF.

2 Preliminaries

Let λ denote the cryptographic security parameter. We assume that all cryptographic algorithms implicitly take 1^λ as input. A function $\mu(\cdot) : \mathbb{N} \rightarrow \mathbb{R}^+$ is said to be negligible if for any polynomial $\text{poly}(\cdot)$ there exists λ_0 such that for all $\lambda > \lambda_0$ we have $\mu(\lambda) < \frac{1}{\text{poly}(\lambda)}$. We will use $\text{negl}(\cdot)$ to denote an unspecified negligible function and $\text{poly}(\cdot)$ to denote an unspecified polynomial function.

Let X and Y be two distributions. Then i) $X \approx_c Y$ means that the distributions are computationally indistinguishable, ii) $X \approx_\varepsilon Y$ means that their statistical distance is at

most ε and, iii) $X \approx_s Y$ means that their statistical distance is at most negl in λ . For a finite set S , we denote $x \leftarrow_{\S} S$ as the process of sampling x uniformly from the set S .

For a string $\mathbf{s} = (s_1, \dots, s_k)$ of size k , we use $\mathbf{s}_{\leq i}$ to denote the string $(s_1, \dots, s_i)$ of size i .

2.1 Cryptographic Primitives

In this work, we need the following cryptographic primitives.

Universal One-Way Hash. We recall the definition of universal one-way hash functions.

Definition 1 (Universal One-Way Hash). *A collection of function families $\mathcal{H} = \{H_\lambda\}_{\lambda \in \mathbb{N}}$, where each H_λ is a function family*

$$H_\lambda = \{h_k : \{0, 1\}^\ell \rightarrow \{0, 1\}^m\}_{k \in \{0, 1\}^\lambda}$$

is a universal one-way hash family if it fulfills the following properties:

- (Efficiency) *For all k , computing $h_k(x)$ takes polynomial time.*
- (Compression) *The functions $h_k(x)$ should be compressing, that is, $m < \ell$.*
- (Universal One-Wayness) *There exists a negligible function negl such that for all PPT adversaries $\mathcal{A}$ we have that the success probability in the following experiment is bounded by $\text{negl}(\lambda)$:*
 - *The adversary sends $x \in \{0, 1\}^\ell$.*
 - *Sample a key $k \leftarrow_{\S} \{0, 1\}^\lambda$ uniformly at random and send it to the adversary.*
 - *The adversary sends $y \in \{0, 1\}^\ell$ and wins if $h_k(x) = h_k(y)$.*

Lemma 1 ([Rom90, KK05]). *If one-way functions exist then universal one-way hash functions exist.*

We can easily build a Merkle hash tree from universal one-way hash functions. The security property of the new Merkle hash function is the same as the original hash, that is, the one in Definition 1. Moreover, the new Merkle tree hash function has local openings, meaning that opening a single bit of the preimage requires only to reveal a root-to-leaf path.

Garbled Circuits. Garbled circuits were first introduced by Yao [Yao86] and can be constructed from one-way functions.

Definition 2. *Let $\mathcal{C} = \{C : \{0, 1\}^n \rightarrow \{0, 1\}^m\}$ be a class of circuits. A garbled circuit for the class $\mathcal{C}$ is composed by a tuple of algorithms (Garble, Eval) such that:*

- *Garble($1^\lambda, C, \{\text{lab}_{i,b}\}_{i \in [n], b \in \{0,1\}}\)$* takes as input a circuit $C \in \mathcal{C}$ and input labels $\{\text{lab}_{i,b}\}_{i \in [n], b \in \{0,1\}}$. It outputs a garbled circuit $\tilde{C}$.

- $\text{Eval}(\tilde{C}, \{\text{lab}_{i,x_i}\}_{i \in [n]})$ takes as input a garbled circuit $\tilde{C}$ and a garbled input represented as a sequence of input labels $\{\text{lab}_{i,x_i}\}_{i \in [n]}$. It outputs a value y .

A garbled circuit scheme should fulfill the following properties:

- (Correctness) For all circuits $C \in \mathcal{C}$ we have that

$$\Pr \left[C(x) \leftarrow \text{Eval}(\tilde{C}, \{\text{lab}_{i,x_i}\}_{i \in [n]}) \right] = 1$$

where $\tilde{C} \leftarrow \text{Garble}(1^\lambda, C, \{\text{lab}_{i,b}\}_{i \in [n], b \in \{0,1\}})$.

- (Security) There is a PPT simulator Sim such that for any $C \in \mathcal{C}$ and $x \in \{0,1\}^n$, we have that:

$$\left(\tilde{C}, \{\text{lab}_{i,x_i}\}_{i \in [n]} \right) \approx_c \text{Sim} \left(1^\lambda, C(x), \{\text{lab}_i\}_{i \in [n]} \right)$$

where $\text{lab}_{i,b} \leftarrow_{\$} \{0,1\}^\lambda$, $\text{lab}_i \leftarrow_{\$} \{0,1\}^\lambda$, and $\tilde{C} \leftarrow_{\$} \text{Garble}(1^\lambda, C, \{\text{lab}_{i,b}\}_{i \in [n], b \in \{0,1\}})$.

The definition for garbled circuits that we use in this work is slightly different from the one used in previous works, where the input labels are output by the **Garble** algorithm, instead of being given as input to it. However, we note that, as long as the input labels are chosen uniformly at random, they can be given as input to the **Garble** without affecting security.

Commitments. We will use commitments with statistical binding and computational hiding, which can be built from OWF [Nao90].

Definition 3. A commitment scheme is a tuple of algorithms $(\text{Com}, \text{ComOpen})$ such that

- **Com** takes as input a security parameter λ and a value x . It outputs a commitment c and an opening open .
- **ComOpen** takes as input a opening open , a commitment c and a value x . It outputs a bit $b \in \{0,1\}$.

A commitment scheme should fulfill the following properties:

- (Correctness) For all messages x we have that

$$\Pr \left[1 \leftarrow \text{ComOpen}(\text{open}, c, x) : (c, \text{open}) \leftarrow \text{Com}(1^\lambda, x) \right] = 1.$$

- (Statistical binding) For all computationally unbounded adversaries $\mathcal{A}$ we have that

$$\Pr \left[\begin{array}{l} 1 \leftarrow \text{ComOpen}(\text{open}, c, x) \\ 1 \leftarrow \text{ComOpen}(\text{open}', c, x') : (\text{open}, c, x, \text{open}', x') \leftarrow \mathcal{A}(1^\lambda) \\ x \neq x' \end{array} \right] = \text{negl}(\lambda).$$

- (Computational hiding) For all pairs of message m_0, m_1 , we have that

$$\{c : (c, \text{open}) \leftarrow \text{Com}(1^\lambda, x)\} \approx_c \{c' : (c', \text{open}') \leftarrow \text{Com}(1^\lambda, x')\}$$

Pseudorandom Functions. Pseudorandom functions (PRF) are functions for which no adversary is able to distinguish from uniform. PRFs are known to exist assuming one-way functions [GGM86, HILL99].

Definition 4 (Pseudorandom Function). *Let $\alpha = \alpha(\lambda)$ and $\beta = \beta(\lambda)$. A Pseudorandom Function (PRF) is defined by a keyed function $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^\alpha \rightarrow \{0, 1\}^\beta$ such that, for any adversary $\mathcal{A}$*

$$|\Pr[1 \leftarrow \mathcal{A}(y, x) : y \leftarrow \text{PRF}(K, x)] - \Pr[1 \leftarrow \mathcal{A}(y, x) : y \leftarrow f(x)]| \leq \text{negl}[\lambda]$$

for any $x \in \{0, 1\}^\alpha$, where $f : \{0, 1\}^\alpha \rightarrow \{0, 1\}^\beta$ is a uniformly chosen random function and the key K is sampled uniformly at random from $\{0, 1\}^\lambda$.

Multi-Party Computation in the ROT-Model. We work in the standard ideal/real-world paradigm. Let $\mathcal{F}$ be a functionality and π a protocol that implements $\mathcal{F}$. Let $\text{IDEAL}_{\mathcal{F}, \text{Sim}}$ be a random variable that represents the view of adversary Sim after the execution of $\mathcal{F}$. Similarly, let $\text{REAL}_{\pi, \mathcal{A}}^{\mathcal{G}}$ be a random variable that represents the view of $\mathcal{A}$ after the execution of π and with access to the functionality $\mathcal{G}$.

Definition 5. *A protocol π implements $\mathcal{F}$ in the $\mathcal{G}$ -hybrid model if for every PPT adversary $\mathcal{A}$ there is a PPT simulator Sim such that the distributions $\text{IDEAL}_{\mathcal{F}, \text{Sim}}$ and $\text{REAL}_{\pi, \mathcal{A}}^{\mathcal{G}}$ are computationally indistinguishable.*

Let $\mathcal{C} : \mathcal{X}_1 \times \dot{\times} \mathcal{X}_N \rightarrow \mathcal{Z}$ be a circuit. We define the ideal functionality for MPC that allows two parties to jointly compute $\mathcal{C}(x_1, \dots, x_N)$.

MPC functionality $\mathcal{F}_{\mathcal{C}}$ for $\mathcal{C}$. This implements the following functionality.

- **P_i's input:** $x_i \in \mathcal{X}_i$.
- **Output:** $\mathcal{C}(x_1, \dots, x_N)$.

Moreover, we define the random oblivious transfer functionality $\mathcal{F}_{\text{ROT}}$.

ROT functionality $\mathcal{F}_{\text{ROT}}$. This implements the following functionality.

- **Sample $\mathbf{m}_0, \mathbf{m}_1 \leftarrow_{\$} \{0, 1\}^\lambda$ and $b \leftarrow_{\$} \{0, 1\}$.**
- **Receiver's output:** $(b, \mathbf{m}_b)$.
- **Sender's output:** $(\mathbf{m}_0, \mathbf{m}_1)$.

Theorem 1 ([IPS08]). *Assume OWFs exists. Then there exists a protocol π that implements $\mathcal{F}_{\mathcal{C}}$ against malicious adversaries for any circuit $\mathcal{C}$ in the $\mathcal{F}_{\text{ROT}}$ -hybrid model.*

Secret Sharing. A secret sharing scheme allows to split a secret into shares such that i) the shares individually reveal nothing about the secret, and ii) the secret can be reconstructed from a subset of shares. It is well-known that secret sharing schemes for threshold [Sha79] or monotone policies [BL90] can be built information theoretically.

Definition 6. A secret sharing scheme for the class of functions $\mathcal{P}$ is composed by a tuple algorithms (Share, Reconst) such that

- **Share**(A, s) takes as input an access structure $A \in \mathcal{P}$ and a secret s . It outputs shares $(s_1, \dots, s_N)$.
- **Reconst**($\{s_i\}_{i \in S}, A$) is a deterministic algorithm that takes as input a subset S of shares $\{s_i\}_{i \in S}$ and an access structure. It outputs the secret s .

A secret sharing scheme should fulfill the following properties:

- (Correctness) For all secrets s and all subsets S such that $A(S) = 1$ we have that

$$\Pr[s \leftarrow \text{Reconst}(\{s_i\}_{i \in S}, A)] = 1$$

where $(s_1, \dots, s_N) \leftarrow \text{Share}(A, s)$.

- (Security) For all pairs of messages s_0, s_1 and for all subsets S such that $A(S) = 0$ we have that

$$\{\{s_i\}_{i \in S} : (s_1, \dots, s_N) \leftarrow \text{Share}(A, s_0)\} \approx_s \{\{s_i\}_{i \in S} : (s_1, \dots, s_N) \leftarrow \text{Share}(A, s_1)\}.$$

Digital Signatures. In this work, we use signatures that achieve existential unforgeability. This can be built from OWF [Lam79, NY89, Rom90].

Definition 7. A signature scheme is composed by a tuple of algorithms (KeyGen, Sign, Ver) such that

- **KeyGen**(1^λ) takes as input a security parameter λ . It outputs a verification key $\mathbf{vk}$ and a secret key $\mathbf{sk}$.
- **Sign**($\mathbf{sk}, m$) takes as input a secret key $\mathbf{sk}$ and a message m . It outputs a signature σ .
- **Ver**($\mathbf{vk}, \sigma, m$) takes as input a verification key $\mathbf{vk}$, a signature σ and a message m . It outputs a bit b .

A signature scheme should fulfill the following properties:

- (Correctness) For all m we have that

$$\Pr \left[1 \leftarrow \text{Ver}(\mathbf{vk}, \sigma, m) : \begin{array}{l} (\mathbf{vk}, \mathbf{sk}) \leftarrow \text{KeyGen}(1^\lambda) \\ \sigma \leftarrow \text{Sign}(\mathbf{sk}, m) \end{array} \right] = 1.$$

- (Unforgeability) For all PPT adversarise $\mathcal{A}$ we have that

$$\Pr \left[1 \leftarrow \text{Ver}(\text{vk}, \sigma^*, m^*) : \begin{array}{l} (\text{vk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda) \\ (\sigma^*, m^*) \leftarrow \mathcal{A}^{\text{Sign}(\text{sk}, \cdot)}(\text{vk}) \end{array} \right] = \text{negl}(\lambda)$$

where $\mathcal{A}$ has oracle access to a signing oracle $\text{Sign}(\text{sk}, \cdot)$ and m^* was never queried to it.

In this work, we assume wlog that the signing algorithm is deterministic. This can be achieved by taking any signature scheme and choosing the random coins for the signing algorithm using a PRF.

Zero-Knowledge Proofs of Knowledge. A zero-knowledge proof of knowledge allows a prover to prove to a verifier that a NP statement is in NP without ever revealing the corresponding witness. We will need a strong soundness property called proof of knowledge. This can be achieved from OWF [GMW86].

Definition 8. A zero-knowledge proof of knowledge for an NP language $\mathcal{L}$ with relation $\mathcal{R}$ is an interactive protocol between two algorithms (P, V) that should fulfill the following properties:

- (Completeness) For all $x \in \mathcal{L}$, $\text{V}(x)$ accepts with probability 1 after interacting with $\text{P}(x, w)$.
- (Proof of knowledge) There exists an extractor Ext such that for all malicious P , the probability that $\mathcal{R}(x, w) = 1$ where $w \leftarrow \text{Ext}^{\text{P}^*}(x)$ is at least as high as the probability of V accepting the proof on input x after interacting with P^* .
- (Zero-knowledge) For all malicious V^* There exists a simulator Sim such that

$$\{\text{Sim}(x)\} \approx_c \{\text{view}_{\text{V}^*}(\langle \text{P}(x, w), \text{V}(x)^* \rangle)\}$$

where $\text{view}_{\text{V}^*}(\langle \text{P}(x, w), \text{V}(x)^* \rangle)$ represents the view of V^* after interacting with P .

3 Tree Oblivious Transfer Extension

In this section we present our main protocol TreeOT to synthesize random OT correlation. Before presenting the protocol, we recall the definition of random OT correlations. Let $\lambda \in \mathbb{N}$, we say that a pair of strings:

$$(b, \tilde{m}) \in \{0, 1\} \times \{0, 1\}^\lambda \quad \text{and} \quad (m_0, m_1) \in \{0, 1\}^\lambda \times \{0, 1\}^\lambda$$

is an OT-correlation $((b, \tilde{m}), (m_0, m_1)) \in \mathcal{R}_{\text{OT}}(1^\lambda)$ if $m_b = \tilde{m}$. For convenience, we define the following sampling algorithm $\text{SampOT}(1^\lambda)$ for random OT-correlations:

- For $i \in [\lambda]$ sample $\left((b^{(i)}, \tilde{m}^{(i)}), (m_0^{(i)}, m_1^{(i)}) \right) \leftarrow_{\S} \mathcal{R}_{\text{OT}}(1^\lambda)$.

- Return $\text{ot}_R := \{b^{(i)}, \tilde{m}^{(i)}\}_i$ and $\text{ot}_S := \{m_1^{(i)}, m_0^{(i)}\}_i$.

Then, a **TreeOT** is an interactive protocol between two parties (Alice and Bob) with the following interface:

- (Setup) Sample:

$$(\text{ot}_{R,A}, \text{ot}_{S,B}) \leftarrow_{\$} \text{SampOT}(1^\lambda) \quad (\text{ot}_{R,B}, \text{ot}_{S,A}) \leftarrow_{\$} \text{SampOT}(1^\lambda)$$

and send $(\text{ot}_{R,A}, \text{ot}_{S,A})$ to Alice and $(\text{ot}_{R,B}, \text{ot}_{S,B})$ to Bob.

- (OT Synthesis) On input a string $S \in \{0, 1\}^\ell$, for some polynomial $\ell = \ell(\lambda)$, Alice and Bob engage in an interactive protocol, where the outputs are new OT correlations for Alice $(\text{ot}_{R,A,S}, \text{ot}_{S,A,S})$ and new OT correlations for Bob $(\text{ot}_{R,B,S}, \text{ot}_{S,B,S})$.

Given that the setup is executed once, the OT-synthesis protocol can be run an arbitrary polynomial number of times by Alice and Bob. As for security, we define the following ideal functionality $\mathcal{F}_{\text{TreeOT}}$:

- (Parameters) The ideal functionality takes as input the security parameter 1^λ , and initializes an empty set $Z := \emptyset$.
- (Alice's Input) A string $S_A \in \{0, 1\}^\ell$.
- (Bob's Input) A string $S_B \in \{0, 1\}^\ell$.
- (Output) If $S_A \neq S_B$, return $\perp$. Else denote by $S = S_A = S_B$. If $(S, \cdot) \notin Z$, sample:

$$(\text{ot}_{R,A,S}, \text{ot}_{S,B,S}) \leftarrow_{\$} \text{SampOT}(1^\lambda) \quad (\text{ot}_{R,B,S}, \text{ot}_{S,A,S}) \leftarrow_{\$} \text{SampOT}(1^\lambda)$$

send $(\text{ot}_{R,A,S}, \text{ot}_{S,A,S})$ to Alice and $(\text{ot}_{R,B,S}, \text{ot}_{S,B,S})$ to Bob, and add

$$(S, \text{ot}_{R,A,S}, \text{ot}_{S,A,S}, \text{ot}_{R,B,S}, \text{ot}_{S,B,S}) \mapsto Z.$$

Else if $(S, \cdot) \in Z$ return to Alice and Bob the correlations from the corresponding entry.

We say that **TreeOT** is secure if it fulfills security Definition 5, that is, if it implements $\mathcal{F}_{\text{TreeOT}}$.

3.1 Construction

Let PRF be a PRF. In a slight abuse of notation, we assume that PRF can take inputs of arbitrary polynomial size. Consider the following circuits $\mathcal{C}[S, K]$ which has a string $S \in \{0, 1\}^*$ and a key K hardwired, and $D[S, K']$ which has a string $S \in \{0, 1\}^*$ and a key K' hardwired¹:

¹The output of both of these circuits can be arbitrarily polynomially larger than the inputs.

- $C[S, K](b_1, \dots, b_n, K')$: Compute $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \text{PRF}(K, S) \oplus \text{PRF}(K', S)$ where $\mathbf{m}_0 = (\mathbf{m}_{0,1}, \dots, \mathbf{m}_{0,k})$ and $\mathbf{m}_1 = (\mathbf{m}_{1,1}, \dots, \mathbf{m}_{1,k})$. Compute

$$\mathbf{x} \leftarrow \text{PRF}(K, (\mathbf{b} \| S \| \dagger)) \oplus \text{PRF}(K', (\mathbf{b} \| S \| \dagger))$$

where $\dagger$ is a special symbol.² Output $\{\mathbf{x}, \mathbf{m}_{x_i, i}\}_{i \in [k]}$.

- $D[S, K'](K)$: Output $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \text{PRF}(K, S) \oplus \text{PRF}(K', S)$.

We now describe the main protocol **TreeOT** between two parties P_1 and P_2 on input a string $S = \text{poly}(\lambda)$. The main intuition of the protocol is the following: At each node of the tree, parties do OT extensions. Then, they parse the new extended OT correlations as left and right correlations. They use either the left ones (if the corresponding bit of S is 0) or the right ones (if the corresponding bit of S is 1) for the next iteration.

Setup(1^λ): Sample two OT correlations

$$(\text{OT}_{R,1,2}(\varepsilon), \text{OT}_{S,2,1}(\varepsilon)) \leftarrow_{\$} \text{SampOT}(\ell) \quad \text{and} \quad (\text{OT}_{R,2,1}(\varepsilon), \text{OT}_{S,1,2}(\varepsilon)) \leftarrow_{\$} \text{SampOT}(\ell).$$

Send $(\text{OT}_{R,1,2}(\varepsilon), \text{OT}_{S,1,2}(\varepsilon))$ to P_1 and $(\text{OT}_{R,2,1}(\varepsilon), \text{OT}_{S,2,1}(\varepsilon))$ to P_2 .

TreeOT: For all $0 \leq i < |S|$ do the following:

- P_1 :

- Take as input a PRF key $\hat{K}_1$ which will be used to derive random coins.
- Parse $\text{OT}_{R,1,2}(S_{\leq i})$ as $(\text{ot}_{R,1,2}(S_{\leq i}), \tilde{\text{ot}}_{R,1,2}(S_{\leq i}))$. Additionally, parse

$$\text{ot}_{R,1,2}(S_{\leq i}) = (\text{ot}_{R,1,2,0}(S_{\leq i}), \text{ot}_{R,1,2,1}(S_{\leq i}))$$

and

$$\tilde{\text{ot}}_{R,1,2}(S_{\leq i}) = (\tilde{\text{ot}}_{R,1,2,0}(S_{\leq i}), \tilde{\text{ot}}_{R,1,2,1}(S_{\leq i})).$$

Parse $\text{OT}_{S,1,2}(S_{\leq i})$ as $(\text{ots}_{1,2}(S_{\leq i}), \tilde{\text{ots}}_{1,2}(S_{\leq i}))$. Additionally, parse

$$\text{ots}_{1,2}(S_{\leq i}) = (\text{ots}_{1,2,0}(S_{\leq i}), \text{ots}_{1,2,1}(S_{\leq i}))$$

and

$$\tilde{\text{ots}}_{1,2}(S_{\leq i}) = (\tilde{\text{ots}}_{1,2,0}(S_{\leq i}), \tilde{\text{ots}}_{1,2,1}(S_{\leq i})).$$

- Parse $\text{ot}_{R,1,2,S_{i+1}}(S_{\leq i})$ as $(\mathbf{b}(S_{\leq i}), K_1(S_{\leq i}), \{\mathbf{r}_j(S_{\leq i})\}_j)$, $\text{ots}_{1,2,S_{i+1}}(S_{\leq i})$ as $(\mathbf{t}_0(S_{\leq i}), \mathbf{t}_1(S_{\leq i}))$, $\tilde{\text{ot}}_{R,1,2,S_{i+1}}(S_{\leq i})$ as $(\tilde{\mathbf{b}}(S_{\leq i}), \tilde{K}_1(S_{\leq i}), \{\tilde{\mathbf{r}}_j(S_{\leq i})\}_j)$ and $\tilde{\text{ots}}_{1,2,S_{i+1}}(S_{\leq i})$ as $(\tilde{\mathbf{t}}_0(S_{\leq i}), \tilde{\mathbf{t}}_1(S_{\leq i}))$.

²More precisely, we require that $\dagger$ does not appear in the description of S , in order to avoid collisions for the PRF.

- Compute

$$\hat{D}_{1,2}^{(S_{\leq i})} \leftarrow \text{GC.Garble} \left(D[S_{\leq i}, K_1(S_{\leq i})], (\mathbf{t}_0(S_{\leq i}), \mathbf{t}_1(S_{\leq i})) \right)$$

and

$$\hat{C}_{1,2}^{(S_{\leq i})} \leftarrow \text{GC.Garble} \left(C[S_{\leq i}, \tilde{K}_1(S_{\leq i})], (\tilde{\mathbf{t}}_0(S_{\leq i}), \tilde{\mathbf{t}}_1(S_{\leq i})) \right)$$

where the random coins used to compute these garbled circuits are derived deterministically using a PRF $\text{PRF}(\hat{K}_1, \cdot)$ on input the string $S_{\leq i}$ plus some unique identifier for the round.

- Send $\hat{D}_{1,2}^{(S_{\leq i})}$ and $\hat{C}_{1,2}^{(S_{\leq i})}$ to P_2 .

- P_2 :

- Take as input a PRF key $\hat{K}_2$ which will be used to derive random coins.
- Parse $\text{OT}_{R,2,1}(S_{\leq i})$ as $(\text{ot}_{R,2,1}(S_{\leq i}), \tilde{\text{ot}}_{R,2,1}(S_{\leq i}))$. Additionally, parse

$$\text{ot}_{R,2,1}(S_{\leq i}) = (\text{ot}_{R,2,1,0}(S_{\leq i}), \text{ot}_{R,2,1,1}(S_{\leq i}))$$

and

$$\tilde{\text{ot}}_{R,2,1}(S_{\leq i}) = (\tilde{\text{ot}}_{R,2,1,0}(S_{\leq i}), \tilde{\text{ot}}_{R,2,1,1}(S_{\leq i})).$$

Parse $\text{OT}_{S,2,1}(S_{\leq i})$ as $(\text{ots}_{2,1}(S_{\leq i}), \tilde{\text{ots}}_{2,1}(S_{\leq i}))$. Additionally, parse

$$\text{ots}_{2,1}(S_{\leq i}) = (\text{ots}_{2,1,0}(S_{\leq i}), \text{ots}_{2,1,1}(S_{\leq i}))$$

and

$$\tilde{\text{ots}}_{2,1}(S_{\leq i}) = (\tilde{\text{ots}}_{2,1,0}(S_{\leq i}), \tilde{\text{ots}}_{2,1,1}(S_{\leq i})).$$

- Parse $\text{ot}_{R,2,1,S_{i+1}}(S_{\leq i})$ as $(K_2(S_{\leq i}), \{\mathbf{t}_j(S_{\leq i})\}_j)$, $\text{ots}_{2,1,S_{i+1}}(S_{\leq i})$ as $(\mathbf{r}_0(S_{\leq i}), \mathbf{r}_1(S_{\leq i}))$, $\tilde{\text{ot}}_{R,2,1,S_{i+1}}(S_{\leq i})$ as $(\tilde{\mathbf{b}}(S_{\leq i}), \tilde{K}_2(S_{\leq i}), \{\tilde{\mathbf{r}}_j(S_{\leq i})\}_j)$ and $\tilde{\text{ots}}_{2,1,S_{i+1}}(S_{\leq i})$ as $(\tilde{\mathbf{t}}_0(S_{\leq i}), \tilde{\mathbf{t}}_1(S_{\leq i}))$.
- Compute

$$\hat{D}_{2,1}^{(S_{\leq i})} \leftarrow \text{GC.Garble} \left(D[S_{\leq i}, K_2(S_{\leq i})], (\tilde{\mathbf{r}}_0(S_{\leq i}), \tilde{\mathbf{r}}_1(S_{\leq i})) \right)$$

and

$$\hat{C}_{2,1}^{(S_{\leq i})} \leftarrow \text{GC.Garble} \left(C[S_{\leq i}, \tilde{K}_2(S_{\leq i})], (\mathbf{r}_0(S_{\leq i}), \mathbf{r}_1(S_{\leq i})) \right)$$

where the random coins used to compute these garbled circuits are derived deterministically using a PRF $\text{PRF}(\hat{K}_2, \cdot)$ on input the string $S_{\leq i}$ plus some unique identifier for the round.

- Compute

$$\text{OT}_{S,2,1}(S_{\leq i+1}) \leftarrow \text{GC.Eval} \left(\hat{D}_{1,2}^{(S_{\leq i})}, \text{ot}_{R,2,1,S_{i+1}}(S_{\leq i}) \right)$$

and

$$\text{OT}_{R,2,1}(S_{\leq i+1}) \leftarrow \text{GC.Eval} \left(\hat{C}_{1,2}^{(S_{\leq i})}, \tilde{\text{ot}}_{R,2,1,S_{i+1}}(S_{\leq i}) \right).$$

– Send $\hat{D}_{2,1}^{(S_{\leq i})}$ and $\hat{C}_{2,1}^{(S_{\leq i})}$ to P_1 .

• P_1 : Compute

$$\text{OT}_{S,1,2}(S_{\leq i+1}) \leftarrow \text{GC.Eval}\left(\hat{D}_{2,1}^{(S_{\leq i})}, \text{ot}_{R,2,1,S_{i+1}}(S_{\leq i})\right)$$

and

$$\text{OT}_{R,1,2}(S_{\leq i+1}) \leftarrow \text{GC.Eval}\left(\hat{C}_{2,1}^{(S_{\leq i})}, \tilde{\text{ot}}_{R,1,2,S_{i+1}}(S_{\leq i})\right).$$

3.2 Analysis

We now analyze the scheme **TreeOT** presented above.

Lemma 2 (Correctness). *The scheme TreeOT is correct.*

Proof. For all $0 \leq i < |S|$, we first look at P_1 's output $\text{OT}_{S,1,2}(S_{\leq i+1})$. By the correctness of the garbled circuit scheme, we have that

$$\text{OT}_{S,1,2}(S_{\leq i+1}) = (\mathbf{t}_0(S_{\leq i+1}), \mathbf{t}_1(S_{\leq i+1})) \leftarrow \text{PRF}(K_1(i), S_{\leq i}) \oplus \text{PRF}(K_2(i), S_{\leq i})$$

which is the output of the garbled circuit $\hat{D}_{2,1}^{(S_{\leq i})}$. On the other hand, P_2 outputs $\text{OT}_{R,2,1}(S_{\leq i+1})$ which is the output of the garbled circuit $\hat{C}_{1,2}^{(S_{\leq i})}$. Again, by the correctness of the garbled circuit, we have that

$$\text{OT}_{R,2,1}(S_{\leq i+1}) = (\mathbf{x}(S_{\leq i+1}), \{\mathbf{t}_{x_j}(S_{\leq i+1})\}_j).$$

Hence, $\mathcal{R}_{\text{OT}}(\text{OT}_{S,1,2}(S_{\leq i+1}), \text{OT}_{R,2,1}(S_{\leq i+1})) = 1$. We can use a similar reasoning to argue that $\mathcal{R}_{\text{OT}}(\text{OT}_{S,2,1}(S_{\leq i+1}), \text{OT}_{R,1,2}(S_{\leq i+1})) = 1$ for all S . $\square$

Theorem 2 (Security). *The scheme TreeOT implements the ideal functionality $\mathcal{F}_{\text{TreeOT}}$ against semi-honest adversaries.*

Proof. We have to show that the scheme is (semi-honest) secure both when P_1 or P_2 are corrupted.

Assume that P_1 is corrupted. We start by describing the simulator.

• Sim_{P_1} : Initialize a list V as $\emptyset$. Whenever P_1 initializes a session on input $S = (S_1, \dots, S_\alpha)$, first query the ideal functionality $\mathcal{F}_{\text{TreeOT}}$ on S . Upon receiving $\text{OT}_{R,1,2}(S)$ and $\text{OT}_{S,1,2}(S)$, store $(S, \text{OT}_{R,1,2}(S), \text{OT}_{S,1,2}(S)) \in V$. Additionally do the following:

– For all $0 < i \leq \alpha$, if $(S_{\leq i}, \cdot, \cdot) \notin V$, then sample OT correlations

$$(\text{OT}_{R,1,2}(S_{\leq i}), \text{OT}_{S,2,1}(S_{\leq i})) \leftarrow_{\$} \text{SampOT}(\ell)$$

and

$$(\text{OT}_{R,2,1}(S_{\leq i}), \text{OT}_{S,1,2}(S_{\leq i})) \leftarrow_{\$} \text{SampOT}(\ell)$$

and store $(S_{\leq i}, \text{OT}_{R,1,2}(S_{\leq i}), \text{OT}_{S,1,2}(S_{\leq i})) \in V$.

- Parse $\text{OT}_{R,1,2}(S_{\leq i})$ as $(\text{ot}_{R,1,2}(S_{\leq i}), \tilde{\text{ot}}_{R,1,2}(S_{\leq i}))$. Additionally, parse

$$\text{ot}_{R,1,2}(S_{\leq i}) = (\text{ot}_{R,1,2,0}(S_{\leq i}), \text{ot}_{R,1,2,1}(S_{\leq i}))$$

and

$$\tilde{\text{ot}}_{R,1,2}(S_{\leq i}) = (\tilde{\text{ot}}_{R,1,2,0}(S_{\leq i}), \tilde{\text{ot}}_{R,1,2,1}(S_{\leq i})).$$

- Parse $\text{OT}_{S,1,2}(S_{\leq i})$ as $(\text{ots}_{1,2}(S_{\leq i}), \tilde{\text{ots}}_{1,2}(S_{\leq i}))$. Additionally, parse

$$\text{ots}_{1,2}(S_{\leq i}) = (\text{ots}_{1,2,0}(S_{\leq i}), \text{ots}_{1,2,1}(S_{\leq i}))$$

and

$$\tilde{\text{ots}}_{1,2}(S_{\leq i}) = (\tilde{\text{ots}}_{1,2,0}(S_{\leq i}), \tilde{\text{ots}}_{1,2,1}(S_{\leq i})).$$

- For all $0 < i \leq \alpha$, compute $\hat{D}_{2,1}^{(S_{\leq i-1})} \leftarrow \text{GC.Sim}(\text{OT}_{S,1,2}(S_{\leq i}), \text{ot}_{R,2,1,S_i}(S_{\leq i-1}))$ and $\hat{C}_{2,1}^{(S_{\leq i})} \leftarrow \text{GC.Sim}(\text{OT}_{R,1,2}(S_{\leq i}), \tilde{\text{ot}}_{R,2,1,S_i}(S_{\leq i-1}))$ using always the same random coins.

We now show that the real-world execution is indistinguishable from the simulated one. The proof follows the following sequence of hybrids for all $0 < i \leq |S|$, which is repeated whenever P_1 initializes a session on S .

- Hybrid $\mathcal{H}_0$: This is the real execution of the protocol.
- Hybrid $\mathcal{H}_{i,1}$: In this hybrid, we sample random coins uniformly at random instead of sampling them using a PRF $\text{PRF}(\hat{K}_2, \cdot)$. Whenever the adversary queries on the same $S_{\leq i}$, the simulator uses the same random string to compute the garbled circuits. Indistinguishability of hybrids follows from the security of the PRF.
- Hybrid $\mathcal{H}_{i,2}$: This hybrid is identical to the previous one except that we compute

$$\hat{D}_{2,1}^{(S_{\leq i-1})} \leftarrow \text{GC.Sim}(\text{OT}_{S,1,2}(S_{\leq i}), \text{ot}_{R,2,1,S_i}(S_{\leq i-1}))$$

and

$$\hat{C}_{2,1}^{(S_{\leq i})} \leftarrow \text{GC.Sim}(\text{OT}_{R,1,2}(S_{\leq i}), \tilde{\text{ot}}_{R,2,1,S_i}(S_{\leq i-1})).$$

Indistinguishability of hybrids follows from the security of the garbled circuit.

- Hybrid $\mathcal{H}_{i,3}$: This hybrid is identical to the previous one except that we set $\text{OT}_{S,1,2}(S_{\leq i})$ and $\text{OT}_{R,1,2}(S_{\leq i})$ to be uniformly chosen OT correlations (instead of computing them via a PRF), if $(S_{\leq i}, \cdot, \cdot) \notin V$. Otherwise, if $(S_{\leq i}, \text{OT}_{R,1,2}(S_{\leq i}), \text{OT}_{S,1,2}(S_{\leq i})) \in V$ use $\text{OT}_{S,1,2}(S_{\leq i})$ and $\text{OT}_{R,1,2}(S_{\leq i})$. Indistinguishability of hybrids follows from the pseudo-randomness of the PRF.

Note that the last hybrid corresponds exactly to the simulation and thus the scheme is semi-honestly secure against corrupted P_1 . The proof of security for a corrupted P_2 follows exactly the same reasoning since the protocol is symmetric. $\square$

4 Threshold Signatures from One-Way Functions

4.1 Definition

We present the definition for monotone-policy signatures, Recall that a threshold policy can be expressed as a monotone policy. In this work, we focus on the case of selective security where the adversary announces the set of corrupted parties before seeing the public parameters and the individual secret key shares.

Definition 9. *A monotone-policy signature scheme is a tuple of algorithms $(\text{Setup}, \text{TSign}, \text{TVer})$ such that:*

- $\text{Setup}(1^\lambda, N, A)$ takes as input a security parameter λ , an integer N and a monotone policy A . it outputs a verification key vk and key shares $(s_1, \dots, s_N)$.
- $\text{TSign}(S, \mu)$ is an interactive algorithm between a subset S of parties to sign a message m . It outputs a signature σ .
- $\text{TVer}(\text{vk}, \sigma, \mu)$ takes as input the verification key vk , a signature σ and a message μ . It outputs a bit b .

A monotone-policy signature should fulfill the following properties:

- (Correctness) For all messages μ and all subsets S such that $A(S) = 1$ we have that

$$\Pr \left[1 \leftarrow \text{TVer}(\text{vk}, \sigma, \mu); \begin{array}{l} (\text{vk}, (s_1, \dots, s_N)) \leftarrow \text{Setup}(1^\lambda, N, A) \\ \sigma \leftarrow \text{TSign}(S, \mu) \end{array} \right] = 1.$$

- (Security) For all PPT adversaries we have that

$$\Pr \left[1 \leftarrow \text{TVer}(\text{vk}, \sigma, \mu) : \begin{array}{l} (N, A, S) \leftarrow \mathcal{A}(1^\lambda) \\ (\text{vk}, (s_1, \dots, s_N)) \leftarrow \text{Setup}(1^\lambda, N, A) \\ (\sigma^*, m^*) \leftarrow \mathcal{A}^{\text{TSign}(\cdot, \cdot)}(\text{vk}, \{s_i\}_{i \in S}) \end{array} \right] = \text{negl}(\lambda)$$

where $A(S) = 0$ and m^* was never queried to the signing oracle $\text{TSign}(\cdot, \cdot)$. Notice that $\text{TSign}(\cdot, \cdot)$ is an interactive oracle that emulates the remaining honest parties in the signing protocol.

We only consider selective security in this work, where the adversary announces the corrupted subset of parties at the beginning of the experiment. We also differentiate between *semi-honest* protocols, where the adversary is semi-honest during the signing protocol, and *malicious* protocols, where the adversary behaves maliciously during the signing protocol and tries to extract information from honest parties.

4.2 Semi-Honest Protocol

We first describe a protocol with security against semi-honest adversaries, i.e., the signers are assumed to follow the protocol as specified but can passively try to break the security of the scheme. The protocol relies on the following ingredients:

- A regular digital signature scheme $\text{Sig} = (\text{KeyGen}, \text{Sign}, \text{Ver})$.
- A pseudorandom function PRF.
- A secret sharing scheme $\text{SS} = (\text{Share}, \text{Reconst})$.
- A tree OT extension scheme TreeOT .
- An MPC protocol in the ROT-model that satisfies semi-honest security.

As shown in [Section 2](#), the assumption that such primitives exist is equivalent to the assumption of the existence of one-way functions. For a given set of parties $P_1, \dots, P_N$, the algorithms of our protocol are defined below.

- **Setup**($1^\lambda, N, A$): Sample a key pair $(\text{vk}, \text{sk}) \leftarrow_{\$} \text{KeyGen}(1^\lambda)$, then compute the shares of the secret key $(\text{s}_1, \dots, \text{s}_N) \leftarrow_{\$} \text{Share}(A, \text{sk})$. For all $i, j \in [N]$ such that $i \neq j$, sample:

$$(\text{ot}_{R,i,j}, \text{ot}_{S,j,i}) \leftarrow_{\$} \text{SampOT}(1^\lambda)$$

and for each $i \in [N]$ sample a key $\text{k}_i \leftarrow_{\$} \{0, 1\}^\lambda$. Send to the i -th party P_i the tuple:

$$\zeta_i := (\text{k}_i, \text{s}_i, \{\text{ot}_{R,i,j}, \text{ot}_{S,i,j}\}_{j \in [n]})$$

and define the verification key to be vk .

- **TSign**(S, μ): On input an authorized set of parties S along with a message μ , the parties $\{P_i\}_{i \in S}$ proceed as follows. For all $i, j \in [N]$ such that $i \neq j$, each pair of parties (P_i, P_j) run the TreeOT protocol on input:
 - The string $S \parallel \mu$, uniquely determining the set S and the message μ .
 - The seed correlations $(\text{ot}_{R,i,j}, \text{ot}_{S,j,i})$ and $(\text{ot}_{R,j,i}, \text{ot}_{S,i,j})$.

Each party derives the random coins for the protocol deterministically, by computing $\text{PRF}(\text{k}_i, \cdot)$ on input the string $S \parallel \mu$ plus some unique identifier for the round. The tree OT extension protocol returns the correlations $(\text{ot}_{R,i,j}(S \parallel \mu), \text{ot}_{S,i,j}(S \parallel \mu))$ to P_i and $(\text{ot}_{R,j,i}(S \parallel \mu), \text{ot}_{S,j,i}(S \parallel \mu))$ to P_j .

Let $C_{S,A,\mu}(\{\text{s}_i\}_{i \in S})$ be the circuit that, on input the shares $\{\text{s}_i\}_{i \in S}$, runs the reconstruction algorithm $\text{sk}^* \leftarrow \text{Reconst}(\{\text{s}_i\}_{i \in S}, A)$ and returns $\sigma = \text{Sign}(\text{sk}^*, \mu)$. The parties $\{P_i\}_{i \in S}$ run an MPC protocol for $C_{S,A,\mu}$ using the obtained OT correlations. Once again, the random coins needed for the MPC protocol are derived deterministically, by computing $\text{PRF}(\text{k}_i, \cdot)$ on input the string $S \parallel \mu$ plus some unique identifier for the round. The output of the protocol is defined to be σ .

- $\text{TVer}(\text{vk}, \sigma, \mu)$: Return $\text{Ver}(\text{vk}, \sigma, \mu)$.

The correctness of the protocol follows as a straightforward invocation of the correctness of the underlying cryptographic primitives. Furthermore, note that the verification key, the signature, and the verification procedure are identical to the one of the underlying signature scheme, which in particular means that the signature size and the verification runtime are independent of N . The following theorem establishes the security of our construction.

Theorem 3 (Semi-Honest Security). *If one-way functions exists, then the protocol as described above is secure against semi-honest signers.*

Proof. We consider the following sequence of hybrids.

- Hybrid $\mathcal{H}_0$: The simulator computes the protocol transcript as specified in the scheme, for each signing query issued by the distinguisher.
- Hybrid $\mathcal{H}_1$: For all honest parties $\{P_i\}_i$, instead of computing the random coins needed by the protocol using $\text{PRF}(k_i, \cdot)$, the simulator uses truly random coins. If the PRF is inoked again on the same input, the simulator uses the same random string. Indistinguishability follows by a reduction against the pseudorandomness of PRF.
- Hybrid $\mathcal{H}_2$: For all pairs of parties $\{P_i, P_j\}_{i \neq j}$ such that P_i or P_j is honest, simulate the transcript of the calls to the **TreeOT** protocol and program randomly sampled OT correlations. Indistinguishability follows by a reduction against the simulatability of the **TreeOT** protocol.

Next, for each signing query i , we consider a series of sub-hybrids in sequence.

- Hybrid $\mathcal{H}_{i,0}$: The simulator computes the transcript of the i -th signing protocol as in the previous hybrid.
- Hybrid $\mathcal{H}_{i,1}$: The simulator computes $\sigma_i \leftarrow \text{Sign}(\text{sk}, \mu_i)$ and simulates the transcript of the i -th run of the MPC protocol, programming σ_i as the output. Note that at this point (i) all OT correlations are honestly generated, (ii) OT correlations are never reused across signing queries for different messages, and (iii) signing queries for the same message μ_i result into an identical transcript. Therefore each OT correlation is used at most once and indistinguishability follows from the simulatability of the MPC protocol.

Finally, in the last hybrid experiment, security follows from a reduction to the unforgeability of the underlying digital signature scheme. On input vk , the reduction simulates the shares of the corrupted parties (say, by secret sharing the 0 string) and otherwise it proceeds as in the last hybrid. For each signing query, σ_i is computed by querying the signing oracle (provided by the unforgeability experiment) and then programmed as the output of the MPC. The reduction returns the forgery of the adversary as its own forgery. It is easy to see that the reduction perfectly simulates the view of the adversary, so the advantage in breaking the security of the threshold signature equals the advantage in breaking the security of the underlying signature scheme. This is a contradiction and it concludes our proof. $\square$

4.3 Malicious Security

We show how to make the protocol described in [Section 4.2](#) secure against malicious signers. Our main observation is that the protocol is completely deterministic, once we fix the private information ζ_i of party P_i . In particular, this means that each message sent from party P_i to party P_j is a deterministic function of the current protocol transcript τ and party P_i 's internal state, i.e., it is computed as $\text{NextMsg}(\tau, i, j, \zeta_i)$.

We will additionally use the following ingredients:

- A commitment scheme Com .
- A universal one-way hash function $\mathcal{H}$.
- A zero-knowledge proof of knowledge protocol.

Once again, all of these objects can be constructed assuming the existence of one-way function. We then describe the modified protocol in the following.

- **Setup**($1, N, A$): Same as in [Section 4.2](#), except that for all $i \in [N]$, we compute:

$$(c_i, \text{open}_i) \leftarrow_{\$} \text{Com}(\zeta_i).$$

Furthermore we sample a key $k \leftarrow_{\$} \{0, 1\}^\lambda$, compute a Merkle hash:

$$h \leftarrow h_k(c_1, \dots, c_N)$$

and denote by p_i the path of siblings corresponding to the opening of c_i . In addition to ζ_i , each party P_i is given $(c_i, \text{open}_i, p_i)$, and the verification key vk is augmented with the hash h .

- **TSign**(S, μ): Same as in [Section 4.2](#), except that, for every message m sent from P_i to P_j , the two parties engage in a zero-knowledge protocol where P_i plays the role of the prover and P_j plays the role of the verifier, for the following statement. Let τ be the transcript so far, the statement is defined as:

$$\left\{ \begin{array}{l} \exists(\zeta_i, \text{open}_i, c_i, p_i) : \\ m = \text{NextMsg}(\tau, i, j, \zeta_i) \ \& \ \text{ComOpen}(\text{open}_i, c_i, \zeta_i) = 1 \ \& \ \text{HashOpen}(h, p_i, c_i) = 1 \end{array} \right\}.$$

The rest of the protocol proceeds as follows, except that if any of the proofs does not verify, the protocol immediately aborts.

- **TVer**(vk, σ, μ): Same as in [Section 4.2](#).

The correctness of the protocol is immediate. Note that the verification key additionally contains the hash h of the private information, but this does not affect its asymptotic size. We now prove that this transformation lifts the security of the protocol from semi-honest to malicious.

Theorem 4 (Malicious Security). *If one-way functions exists, then the protocol as described above is secure against malicious signers.*

Proof. First, observe that, ignoring the messages of the ZK proofs, the honest transcript of the protocol for every possible message μ is fully determined by the verification key $\mathbf{vk}$ and by the information $\{\zeta_i\}_i$ sent to all parties. This allows us to define two cases.

- Case I: All messages of the protocol (except for the ZK proofs) are identical to the honest transcript.
- Case II: There exists at least a message sent by the corrupted party that differs from the honest transcript.

First we claim that, if the second case happens, then all honest parties immediately abort, except with negligible probability. Assume not, then we can run the knowledge extractor on the corresponding ZK proof who, with at least inverse-polynomial probability, returns a valid witness $(\zeta_i^*, \text{open}_i^*, c_i^*, p_i^*)$. Note that it must be the case that $\zeta_i^* \neq \zeta_i$, since otherwise the message of the protocol would have been identical to the honest one, as it is computed by calling $\text{NextMsg}(\tau, i, j, \zeta_i = \zeta_i^*)$. Thus, we are left with two options:

- $c_i^* = c_i$: In this case we have that:

$$\text{ComOpen}(\text{open}_i^*, c_i, \zeta_i^*) = 1 \quad \text{and} \quad \text{ComOpen}(\text{open}_i, c_i, \zeta_i) = 1$$

contradicting the binding property of the commitment.

- $c_i^* \neq c_i$: In this case we have that:

$$\text{HashOpen}(h, p_i^*, c_i^*) = 1 \quad \text{and} \quad \text{HashOpen}(h, p_i, c_i) = 1$$

contradicting the universal one-wayness of the hash.

Since the second case is always followed by an immediate abort, then we have that all messages sent by the honest parties are identical to the ones specified by the honest transcript (except with negligible probability and except for the aborts). Thus, security reduces to the first case. In the first case, we can define a simulator that simulates all of the ZK proofs of the honest parties, then it simulates the protocol transcript as in the proof of [Theorem 3](#). This simulation is indistinguishable by the zero-knowledge property of the ZK proofs, and by [Theorem 3](#). Security follows by a straightforward reduction against the unforgeability of the signature scheme, along the same lines as the proof of [Theorem 3](#). $\square$

Acknowledgments

Pedro Branco and Giulio Malavolta are supported by the European Research Council through an ERC Starting Grant (Grant agreement No. 101077455, ObfusQation). Pedro Branco is

also supported by FONDAZIONE CARIPLO through the project “Trustless - How to Remove Trust Assumptions from Cryptographic Primitives” (Rif. 2025-0813 ex 2024-1165). Giulio Malavolta is also supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy - EXC 2092 CASA – 390781972.

References

- [AB23] Svetlana Abramova and Rainer Böhme. Anatomy of a high-profile data breach: Dissecting the aftermath of a crypto-wallet case. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023*, pages 715–732. USENIX Association, August 2023. pages 1
- [ASY22] Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-optimal lattice-based threshold signatures, revisited. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *ICALP 2022*, volume 229 of *LIPICs*, pages 8:1–8:20. Schloss Dagstuhl, July 2022. pages 1
- [BBK⁺23] Zvika Brakerski, Maya Farber Brodsky, Yael Tauman Kalai, Alex Lombardi, and Omer Paneth. SNARGs for monotone policy batch NP. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 252–283. Springer, Cham, August 2023. pages 3
- [BCJP24] Maya Farber Brodsky, Arka Rai Choudhuri, Abhishek Jain, and Omer Paneth. Monotone-policy aggregate signatures. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part IV*, volume 14654 of *LNCS*, pages 168–195. Springer, Cham, May 2024. pages 2, 3
- [Bea96] Donald Beaver. Correlated pseudorandomness and the complexity of private computations. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC ’96, page 479–488, New York, NY, USA, 1996. Association for Computing Machinery. pages 3, 4
- [BGG⁺18] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 565–596. Springer, Cham, August 2018. pages 1
- [BGJ⁺23] Leemon Baird, Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. Threshold signatures in the multiverse. In *2023 IEEE Symposium on Security and Privacy*, pages 1454–1470. IEEE Computer Society Press, May 2023. pages 1

- [BL90] Josh Cohen Benaloh and Jerry Leichter. Generalized secret sharing and monotone functions. In Shafi Goldwasser, editor, *CRYPTO'88*, volume 403 of *LNCS*, pages 27–35. Springer, New York, August 1990. pages 9
- [BLS01] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 514–532. Springer, Berlin, Heidelberg, December 2001. pages 1
- [BLT⁺24] Renas Bacho, Julian Loss, Stefano Tessaro, Benedikt Wagner, and Chenzhi Zhu. Twinkle: Threshold signatures from DDH with full adaptive security. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 429–459. Springer, Cham, May 2024. pages 1, 2
- [Bol03] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003*, volume 2567 of *LNCS*, pages 31–46. Springer, Berlin, Heidelberg, January 2003. pages 1
- [CATZ25] Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Partially non-interactive two-round lattice-based threshold signatures. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 268–302, Singapore, 2025. Springer Nature Singapore. pages 1
- [CKGW23] Deirdre Connolly, Chelsea Komlo, Ian Goldberg, and Christopher A Wood. Two-round threshold schnorr signatures with FROST. Technical report, Internet-Draft draft-irtf-cfrg-frost-10, Internet Engineering Task Force, 2023. pages 1
- [CKM23] Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023. pages 1, 2
- [Des88] Yvo Desmedt. Society and group oriented cryptography: A new concept. In Carl Pomerance, editor, *CRYPTO'87*, volume 293 of *LNCS*, pages 120–127. Springer, Berlin, Heidelberg, August 1988. pages 1
- [DF90] Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 307–315. Springer, New York, August 1990. pages 1
- [GGM86] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *J. ACM*, 33(4):792–807, August 1986. pages 8
- [GGN16] Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan. Threshold-optimal DSA/ECDSA signatures and an application to bitcoin wallet security. In Mark

- Manulis, Ahmad-Reza Sadeghi, and Steve Schneider, editors, *ACNS 16International Conference on Applied Cryptography and Network Security*, volume 9696 of *LNCS*, pages 156–174. Springer, Cham, June 2016. pages 1
- [GJM⁺24] Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. hints: Threshold signatures with silent setup. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3034–3052, 2024. pages 1
- [GMW86] Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity and a methodology of cryptographic protocol design (extended abstract). In *27th FOCS*, pages 174–187. IEEE Computer Society Press, October 1986. pages 10
- [HILL99] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4):1364–1396, 1999. pages 8
- [IKNP03] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 145–161. Springer, Berlin, Heidelberg, August 2003. pages 3
- [IPS08] Yuval Ishai, Manoj Prabhakaran, and Amit Sahai. Founding cryptography on oblivious transfer - efficiently. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 572–591. Springer, Berlin, Heidelberg, August 2008. pages 3, 8
- [IR89] Russell Impagliazzo and Steven Rudich. Limits on the provable consequences of one-way permutations. In *21st ACM STOC*, pages 44–61. ACM Press, May 1989. pages 3
- [KG20] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson Jr., and Colin O’Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, Cham, October 2020. pages 1, 2
- [KK05] Jonathan Katz and Chiu-Yuen Koo. On constructing universal one-way hash functions from arbitrary one-way functions. Cryptology ePrint Archive, Report 2005/328, 2005. pages 6
- [Kom24] Chelsea Komlo. Threshold signatures. *IEEE Security & Privacy*, 22(6):85–88, 2024. pages 1
- [KRT24] Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, Cham, August 2024. pages 1

- [Lam79] Leslie Lamport. Constructing digital signatures from a one way function. 1979. pages 1, 9
- [Nao90] Moni Naor. Bit commitment using pseudo-randomness. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 128–136. Springer, New York, August 1990. pages 7
- [NIS] Multi-Party Threshold Cryptography. <https://csrc.nist.gov/projects/threshold-cryptography>. pages 1
- [NWW25] Shafik Nassar, Brent Waters, and David J. Wu. Monotone policy bargs from bargs and additively homomorphic encryption. In Elette Boyle and Mohammad Mahmoody, editors, *Theory of Cryptography*, pages 399–430, Cham, 2025. Springer Nature Switzerland. pages 3
- [NY89] Moni Naor and Moti Yung. Universal one-way hash functions and their cryptographic applications. In *21st ACM STOC*, pages 33–43. ACM Press, May 1989. pages 1, 4, 9
- [Rom90] John Rompel. One-way functions are necessary and sufficient for secure signatures. In *22nd ACM STOC*, pages 387–394. ACM Press, May 1990. pages 1, 6, 9
- [RRJ⁺22] Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, and Dominique Schröder. ROAST: Robust asynchronous schnorr threshold signatures. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2551–2564. ACM Press, November 2022. pages 1, 2
- [Sha79] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, November 1979. pages 9
- [TZ23] Stefano Tessaro and Chenzhi Zhu. Threshold and multi-signature schemes from linear hash functions. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023*, pages 628–658, Cham, 2023. Springer Nature Switzerland. pages 1, 2
- [Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986. pages 6